



UET.CS2043 - Lập trình nâng cao

Refactoring

A case study

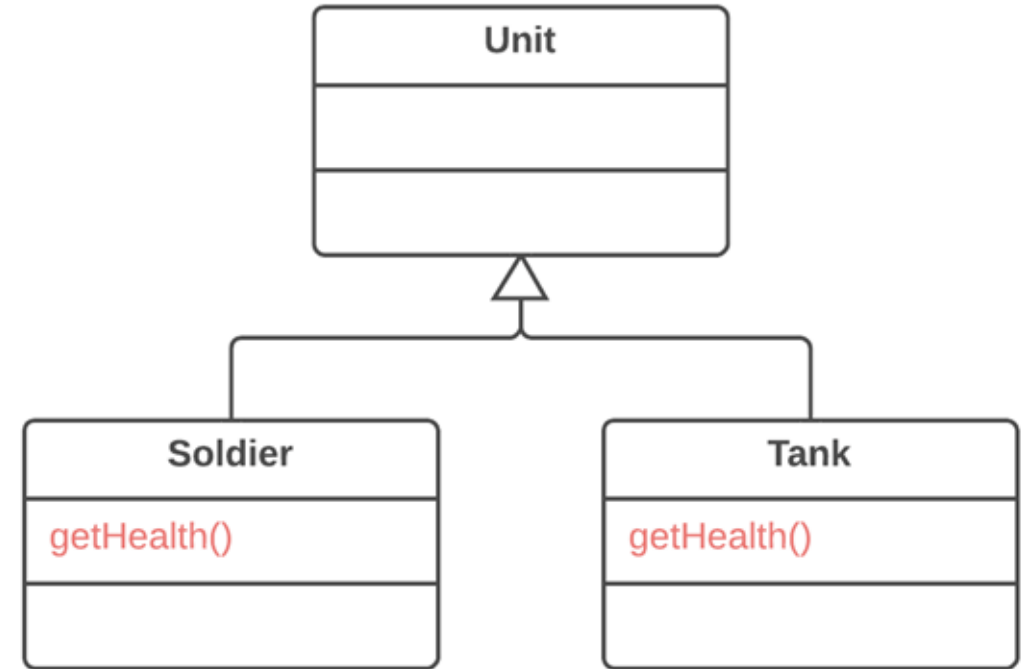
- John reviews a tank game project, examining its architecture and source code. He observes that the **high-level class design** is poorly structured and difficult to maintain.



A case study

Problems

- Some subclasses implement the **same** behavior, but this behavior is **not defined** in the base class.
- Methods defined in the base class are **not suitable** for all subclasses and are therefore frequently overridden.
- The base class defines a method, but subclasses **reimplement** the same behavior using methods with different names.



...



A case study

- John proposed that project management review and clean up the codebase, but the **manager was not enthusiastic**:

“The project is under heavy schedule pressure. We cannot clean up the code right now. Maybe in a few months.”

- John also explained the situation to the developers. The programmers were very interested and clearly recognized the issues.



A case study

- The developers spent a few days **refactoring** the codebase.
 - They eliminated nearly half of the code, making it significantly more **readable**.
 - The team was very satisfied with the outcome and found that adding new classes and using existing ones became much **faster and easier**.



A case study



- Project manager **was not satisfied**
 - Before refactoring, the code was still functioning correctly.
 - The schedule was tight, and there was a significant amount of work to be done.
 - From management's perspective, the software needed to deliver the features requested by the customer, not clean or elegant code.
- The developers had spent two days without contributing to new features that the system was expected to deliver in the coming months.

A case study

- Six months later, the project **failed**, largely because the codebase had become too **complex** to debug or adapt to achieve acceptable performance.

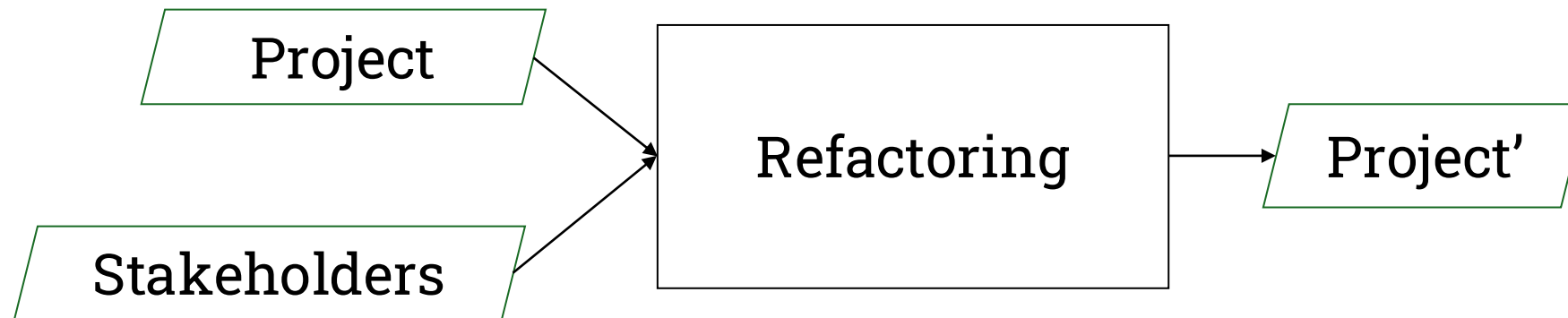


Code smell

- Code smells are usually not bugs
- Identify design deficiencies that negatively impact development speed and elevate the risk of bugs or system failures
- Kind of code smells
 - Condition & Parameter
 - Method
 - Field
 - Class/interface ...

Refactoring

- Refactoring (noun): a change made to the internal structure of software to make it **easier** to understand and cheaper to modify **without** changing its observable behavior



Condition & Parameter

Decompose Conditional

- The longer a piece of code is, the harder it's to understand.

```
if (date.Before(SUMMER_START) || date.After(SUMMER_END))  
    charge = quantity * _winterRate + _winterServiceCharge;  
else  
    charge = quantity * _summerRate;
```



```
if (NotSummer(date))  
    charge = WinterCharge(quantity);  
else  
    charge = SummerCharge(quantity);
```

Long parameter list

```
public void SaveHomeAddress(string name,  
                             string homeAddress,  
                             string country,  
                             string email,  
                             string fileLocation)  
{  
}
```



```
public void SaveHomeAddress(AddressDetails addressDetails)
```

Method refactoring

Extract Method

- **Problem:** You have a code fragment that can be grouped together
- **Why Refactor:** The more lines found in a method, the harder it's to figure out what the method does
- **Solution:**
 - Move this code to a separate new method (or function)
 - Replace the old code with a call to the method.

```
public void PrintClient(Client client) {  
    PrintBanner();  
    // print client details  
    Console.WriteLine("Name: " + client.Name);  
    Console.WriteLine("Address: " + client.Address);  
}
```



```
public void PrintClient(Client client) {  
    PrintBanner();  
    PrintClientDetails(client);  
} 1  
  
private void PrintClientDetails(Client client) {  
    Console.WriteLine("Name: " + client.Name);  
    Console.WriteLine("Address: " + client.Address);  
} 2
```

Inline Method

- Problem: a method body is more obvious than the method itself
- Solution: Replace calls to the method with the method's content and delete the method itself
- Benefit: By minimizing the number of unneeded methods, you make the code more straightforward.

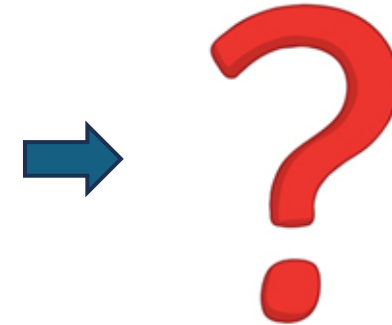
```
1  class PizzaDelivery {  
    1 usage  
2      int numberOfLateDeliveries;  
3      int getRating() {  
4          return moreThanFiveLateDeliveries() ? 2 : 1;  
5      }  
    1 usage  
6      boolean moreThanFiveLateDeliveries() {  
7          return numberOfLateDeliveries > 5;  
8      }  
}
```



```
1  class PizzaDelivery {  
    1 usage  
2      int numberOfLateDeliveries;  
3      int getRating() {  
4          return numberOfLateDeliveries > 5 ? 2 : 1;  
5      }  
6      boolean moreThanFiveLateDeliveries() {  
7          // return numberOfLateDeliveries > 5;  
8      }  
9  }
```


Inline Method - Exercise

```
abstract class Shape
{
    public void Draw()
    {
        try { /*draw*/ }
        catch (Exception e)
        {
            LogError(e);
        }
    }
    private static void LogError(Exception e)
    {
        File.WriteAllText(@"c:\Errors\Exception.txt", e.ToString());
    }
}
```



Inline Method - Exercise

```
abstract class Shape
{
    public void Draw()
    {
        try { /*draw*/ }
        catch (Exception e)
        {
            LogError(e);
        }
    }
}
```

```
private static void LogError(Exception e)
{
    File.WriteAllText(@"c:\Errors\Exception.txt", e.ToString());
}
}
```

```
abstract class Shape
{
    public void Draw()
    {
        try { /*draw*/ }
        catch (Exception e)
        {
            File.WriteAllText(@"c:\Errors\Exception.txt", e.ToString());
        }
    }
}
```

Rename method

- Method name does not show its intention

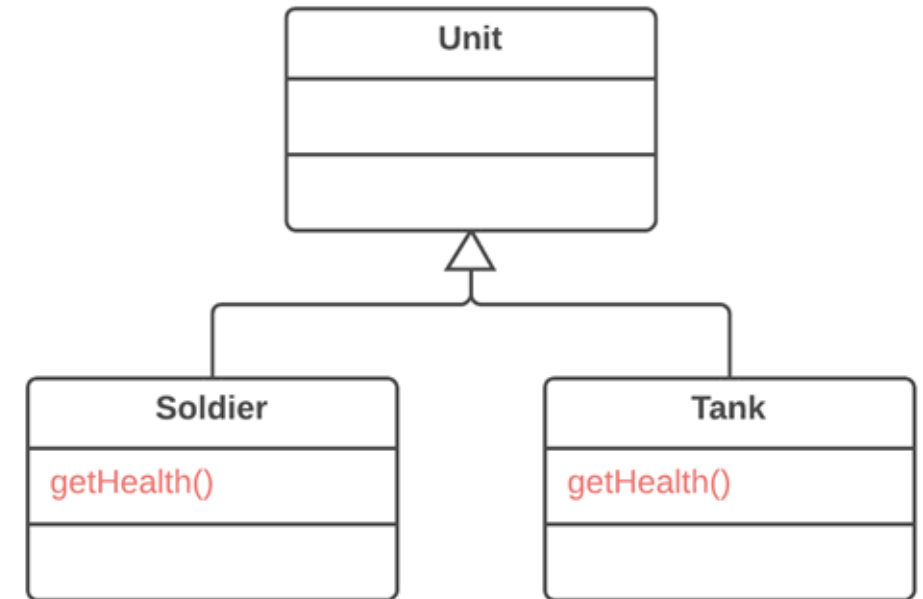
SecurityPrice
+LowerLimitExceed()



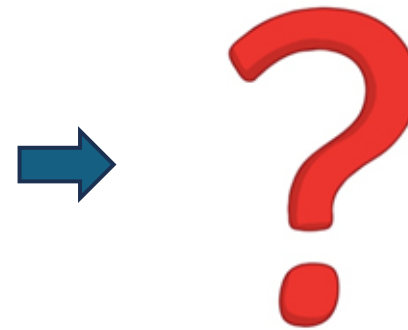
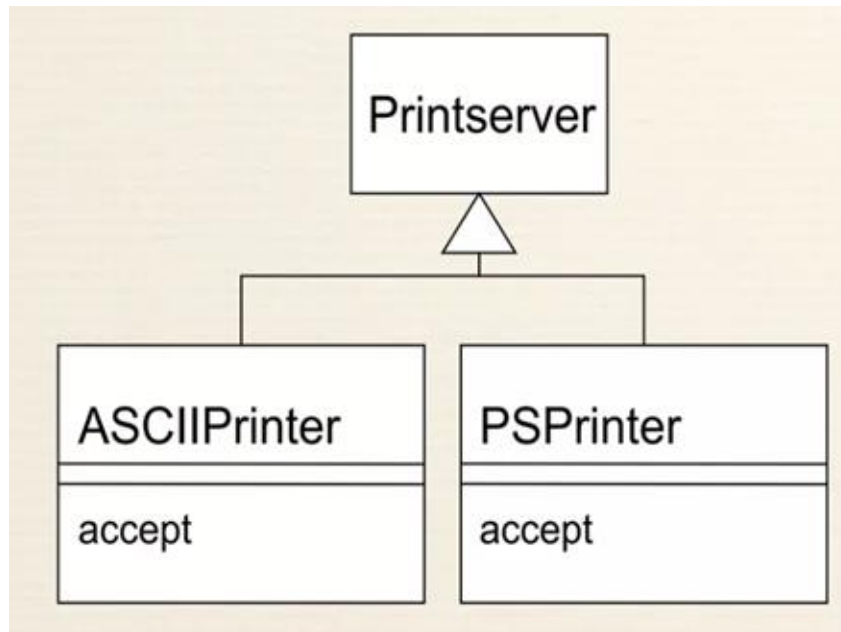
SecurityPrice
+IsLowerLimitExceeded()

Pull up method

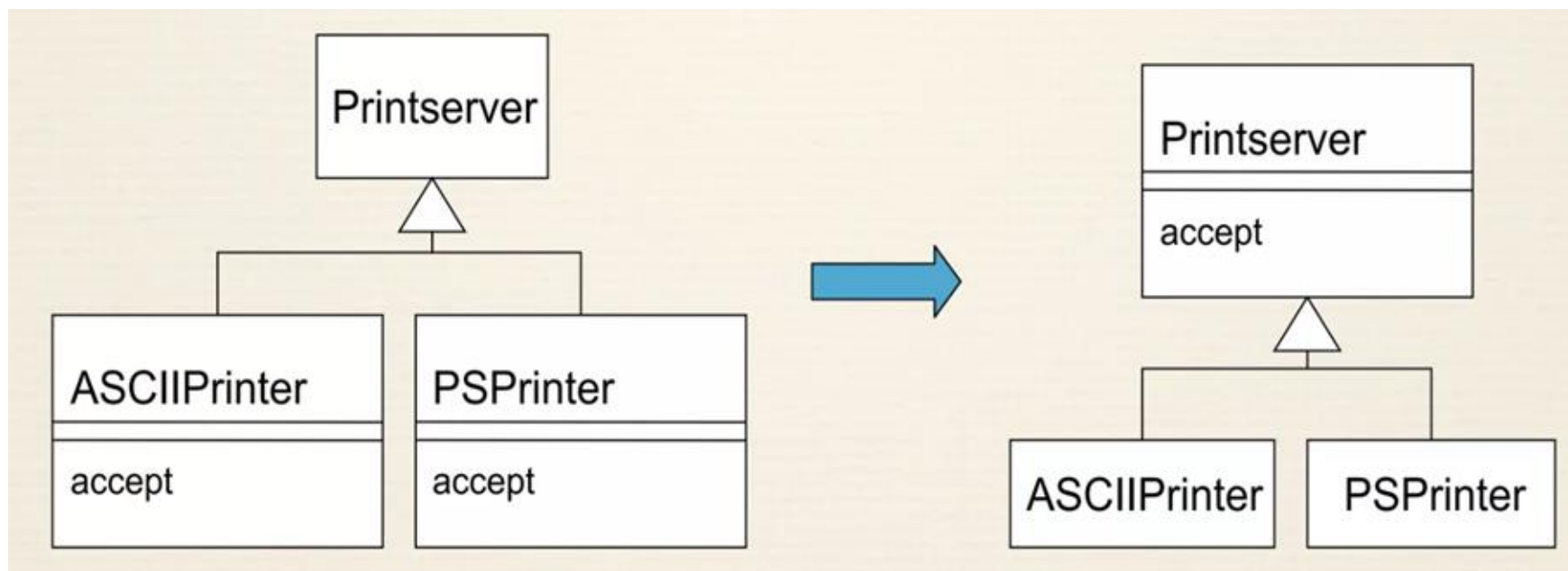
- **Problem:** Your subclasses have methods that perform similar work
- **Solution:** Make the methods identical and then move them to the relevant superclass
- **Why Refactor:** Subclasses grew and developed independently of one another, causing identical (or nearly identical) fields and methods.



Pull up method – Exercise



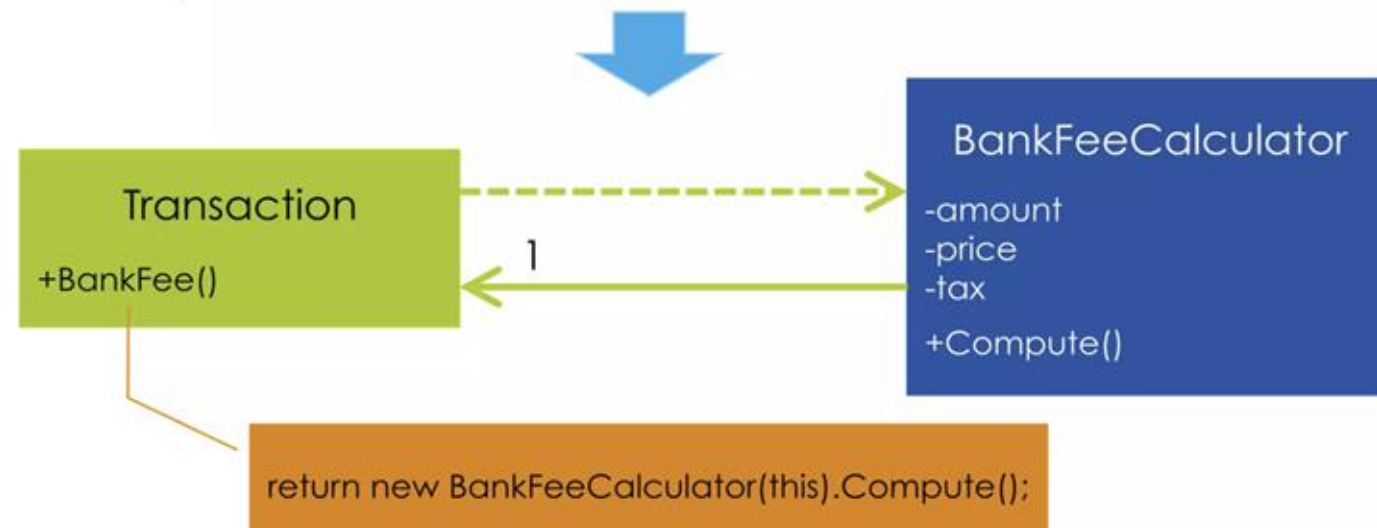
Pull up method – Exercise



Replace method with method objects

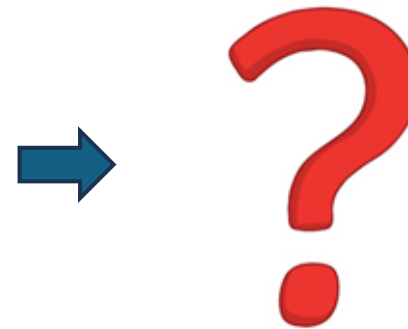
- **Problem:** You have a **long method**
- **Solution**
 - Transform the method into a separate class so that the local variables become fields of the class
 - Then you can split the method into several methods within the same class
- **Drawback:** Another class is added, increasing the overall complexity of the program.

```
class Transaction {  
    public double BankFee() {  
        double amount;  
        double price;  
        double tax;  
        // long computations  
    }  
}
```



Replace method with method objects - Exercise

```
class Order {  
    // ...  
    public double price() {  
        double primaryBasePrice;  
        double secondaryBasePrice;  
        double tertiaryBasePrice;  
        // Perform long computation.  
    }  
}
```



Replace method with method objects - Exercise

```
class Order {  
    // ...  
    public double price() {  
        double primaryBasePrice;  
        double secondaryBasePrice;  
        double tertiaryBasePrice;  
        // Perform long computation.  
    }  
}
```



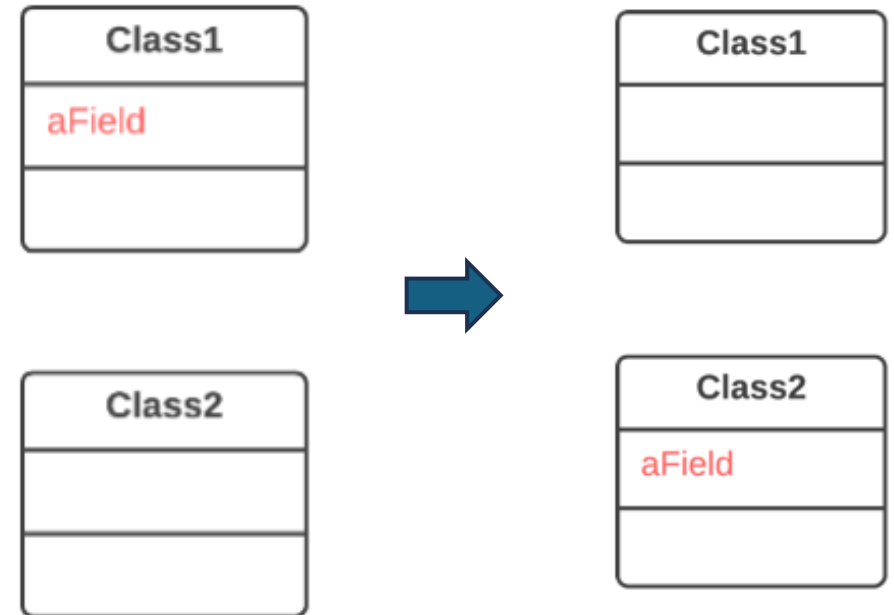
```
class Order {  
    // ...  
    public double price() {  
        return new PriceCalculator(this).compute();  
    }  
}
```

```
class PriceCalculator {  
    private double primaryBasePrice;  
    private double secondaryBasePrice;  
    private double tertiaryBasePrice;  
  
    public PriceCalculator(Order order) {  
        // Copy relevant information from the  
        // order object.  
    }  
  
    public double compute() {  
        // Perform long computation.  
    }  
}
```

Field refactoring

Move Field

- **Problem:** A field is used more in another class than in its own class
- **Solution:** Create a field in a new class and redirect all users of the old field to it



Replace magic number with constant

- **Problem:** Your code uses a number that has a certain meaning to it
- **Solution:** Replace this number with a constant that has a human-readable name explaining the meaning of the number

```
double PotentialEnergy(double mass, double height) {  
    return mass * 9.81 * height;  
}
```



```
static const double GRAVITATIONAL_CONSTANT = 9.81;  
  
double PotentialEnergy(double mass, double height) {  
    return mass * GRAVITATIONAL_CONSTANT * height;  
}
```

Encapsulate fields

- Create and use getter/setter to access field

```
//File Class.java
public class Class {
    public String aString;
}
```

```
//File AnotherClass.java
public class AnotherClass {
    public Class aClass;
    public void method() {
        aClass.aString="string";
    }
}
```



```
//File Class.java
public class Class {
    private String aString;
    public void setaString(String aString) {
        this.aString = aString;
    }
    public String getaString() {
        return aString;
    }
}
```

```
//File AnotherClass.java
public class AnotherClass {
    public Class aClass;
    public void method() {
        aClass.setaString("string");
    }
}
```

Encapsulate fields - Exercise

```
public class NormalFieldClass {  
    public String name;  
  
    public static void main(String[] args)  
    {  
        NormalFieldClass example1 = new NormalFieldClass();  
        example1.name = "myName";  
        System.out.println("My name is " + example1.name);  
    }  
}
```



```
public class EncapsulatedFieldClass {  
    private String name;  
  
    public String getName()  
    {  
        return name;  
    }  
  
    public void setName(String newName)  
    {  
        name = newName;  
    }  
  
    public static void main(String[] args)  
    {  
        EncapsulatedFieldClass example1 = new EncapsulatedFieldClass();  
        example1.setName("myName");  
        System.out.println("My name is " + example1.getName());  
    }  
}
```

Explaining Variable

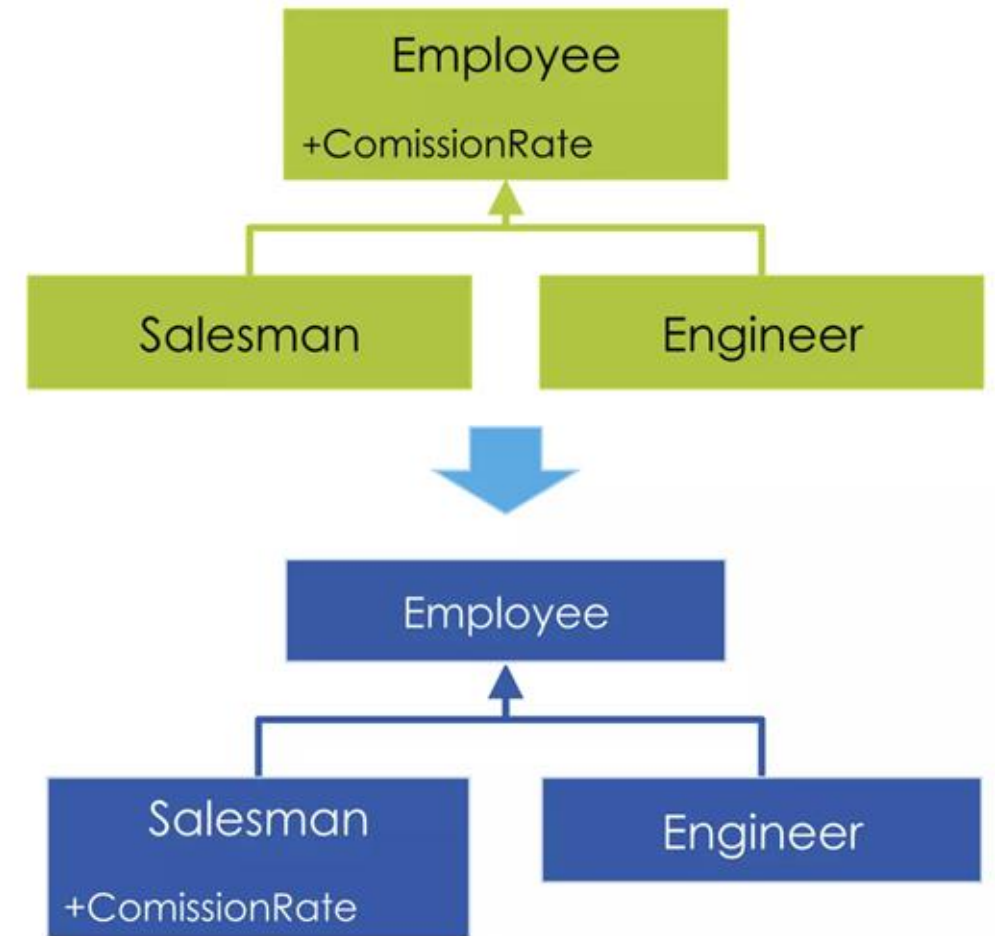
```
if ((platform.toUpperCase().indexOf("MAC") > -1) &&  
    (browser.toUpperCase().indexOf("IE") > -1) &&  
    isInitialized() && resize > 0) {  
    // do something  
}
```



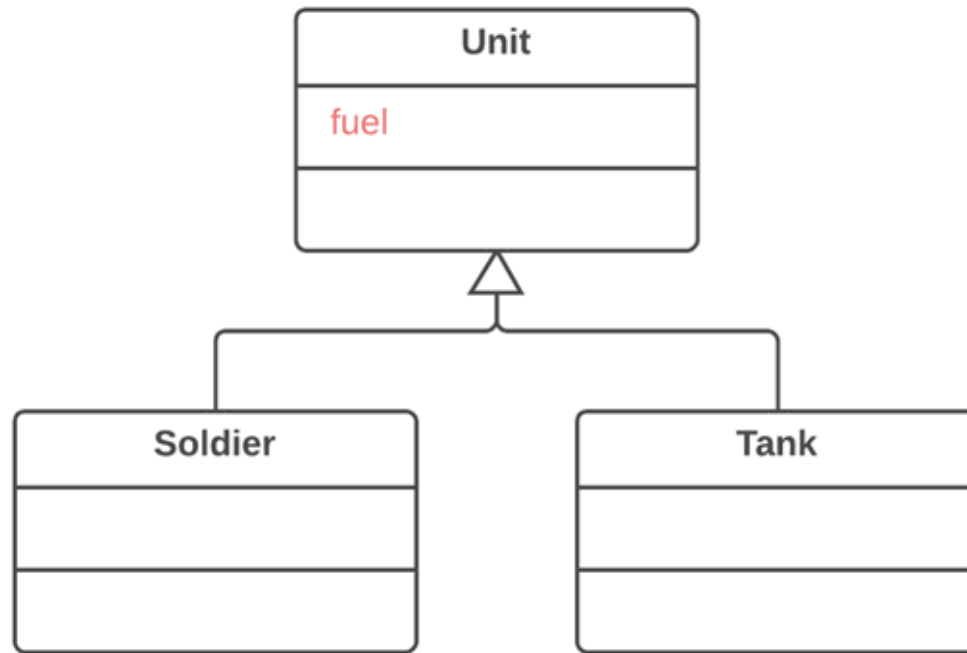
```
var isMac = platform.toUpperCase().indexOf("MAC") > -1;  
var isIEBrowser = browser.toUpperCase().indexOf("IE") > -1;  
var isResized = resize > 0;  
if (isMac && isIEBrowser && isInitialized() && isResized){  
    // do something  
}
```

Push down field

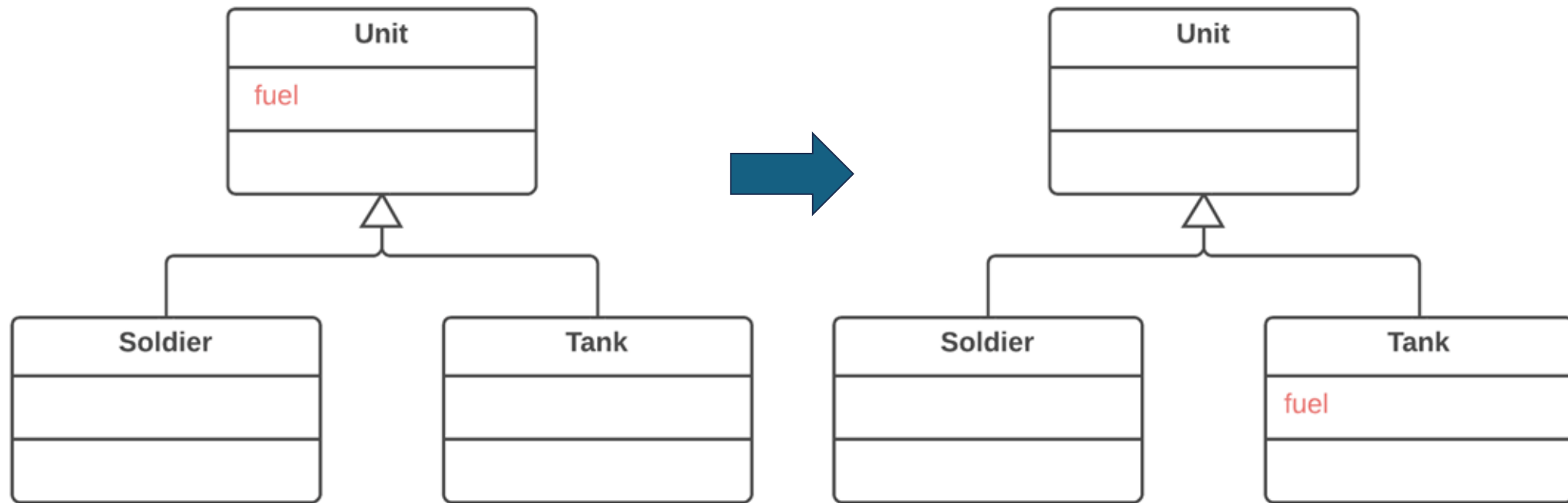
- **Problem:** Is a field used only in a few subclasses?
- **Solution:** Move the field to these subclasses



Push down field - Exercise



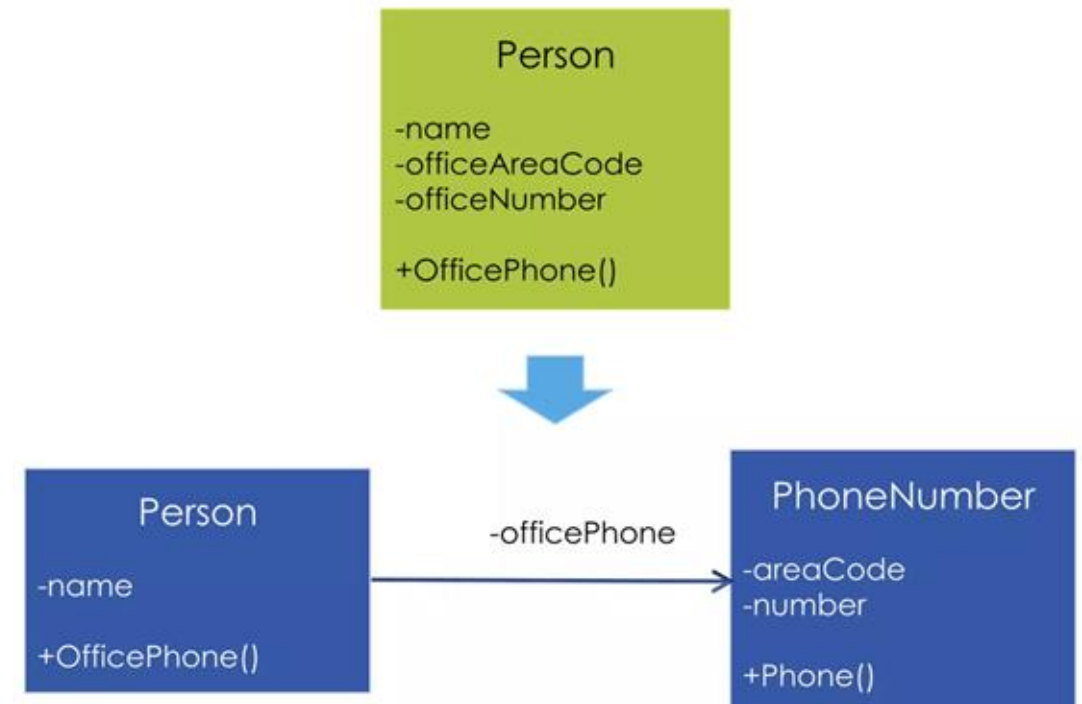
Push down field - Exercise



Class/interface refactoring

Extract Class

- **Problem:** When one class does the work of two, awkwardness results (violate Single Responsibility Principle)
- **Solution:** Instead, create a new class and place the fields and methods responsible for the relevant functionality in it

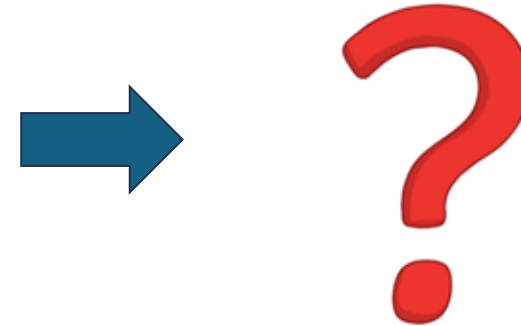


Replace conditional with polymorphism

- **Problem:** You have a conditional that performs various actions depending on object type or properties
- **Solution:** Create subclasses matching the branches of the conditional

Replace conditional with polymorphism

```
class Bird {  
    // ...  
    double getSpeed() {  
        switch (type) {  
            case EUROPEAN:  
                return getBaseSpeed();  
            case AFRICAN:  
                return getBaseSpeed() - getLoadFactor() * numberOfCoccyz;  
            case NORWEGIAN_BLUE:  
                return (isNailed) ? 0 : getBaseSpeed(voltage);  
        }  
        throw new RuntimeException("Should be unreachable");  
    }  
}
```



Replace conditional with polymorphism

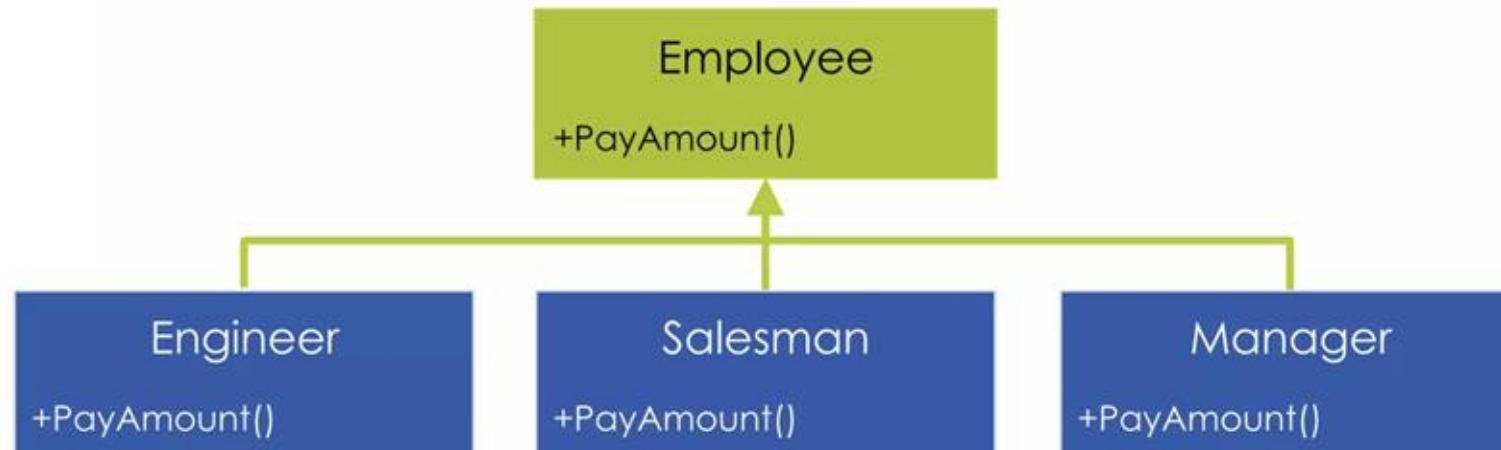
```
class Bird {  
    // ...  
    double getSpeed() {  
        switch (type) {  
            case EUROPEAN:  
                return getBaseSpeed();  
            case AFRICAN:  
                return getBaseSpeed() - getLoadFactor() * numberOfCocoanuts;  
            case NORWEGIAN_BLUE:  
                return (isNailed) ? 0 : getBaseSpeed(voltage);  
        }  
        throw new RuntimeException("Should be unreachable");  
    }  
}
```



```
abstract class Bird {  
    // ...  
    abstract double getSpeed();  
}  
  
class European extends Bird {  
    double getSpeed() {  
        return getBaseSpeed();  
    }  
}  
  
class African extends Bird {  
    double getSpeed() {  
        return getBaseSpeed() - getLoadFactor() * numberOfCoconuts;  
    }  
}  
  
class NorwegianBlue extends Bird {  
    double getSpeed() {  
        return (isNailed) ? 0 : getBaseSpeed(voltage);  
    }  
}  
  
// Somewhere in client code  
speed = bird.getSpeed();
```

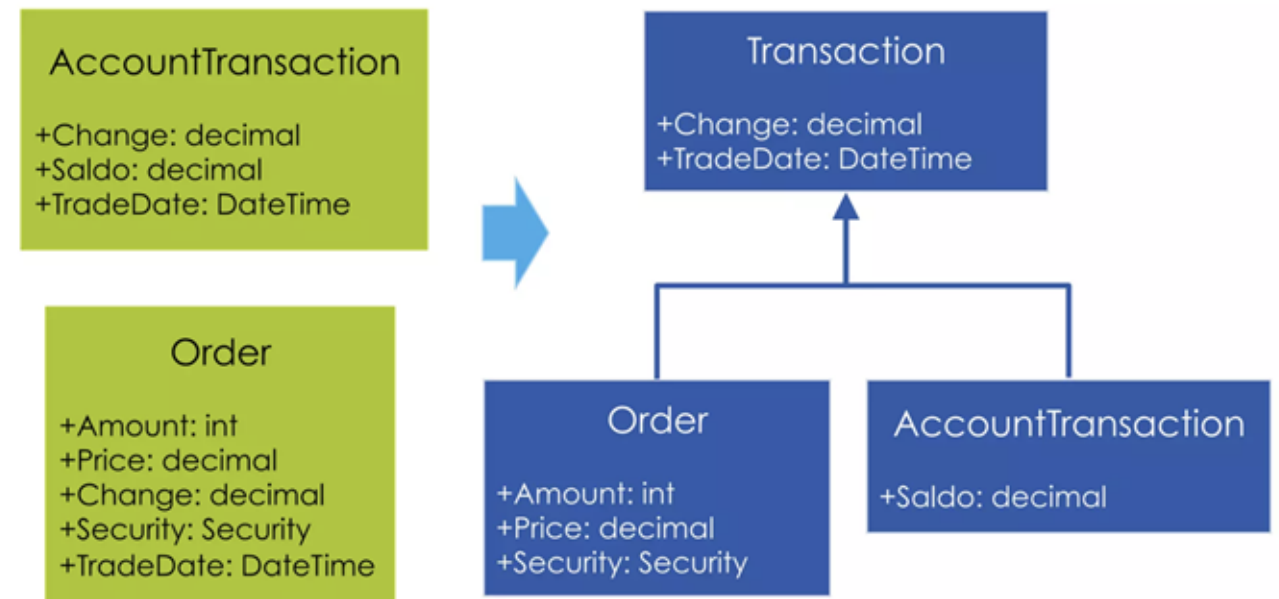
Replace conditional with polymorphism

```
class Employee {  
    public double PayAmount() {  
        switch (_type) {  
            case EmployeeType.Engineer: return _salary;  
            case EmployeeType.Salesman: return _salary + _commission;  
            case EmployeeType.Manager: return _salary + _bonus;  
            default: throw new WrongEmployeeTypeException();  
        }  
    }  
}
```

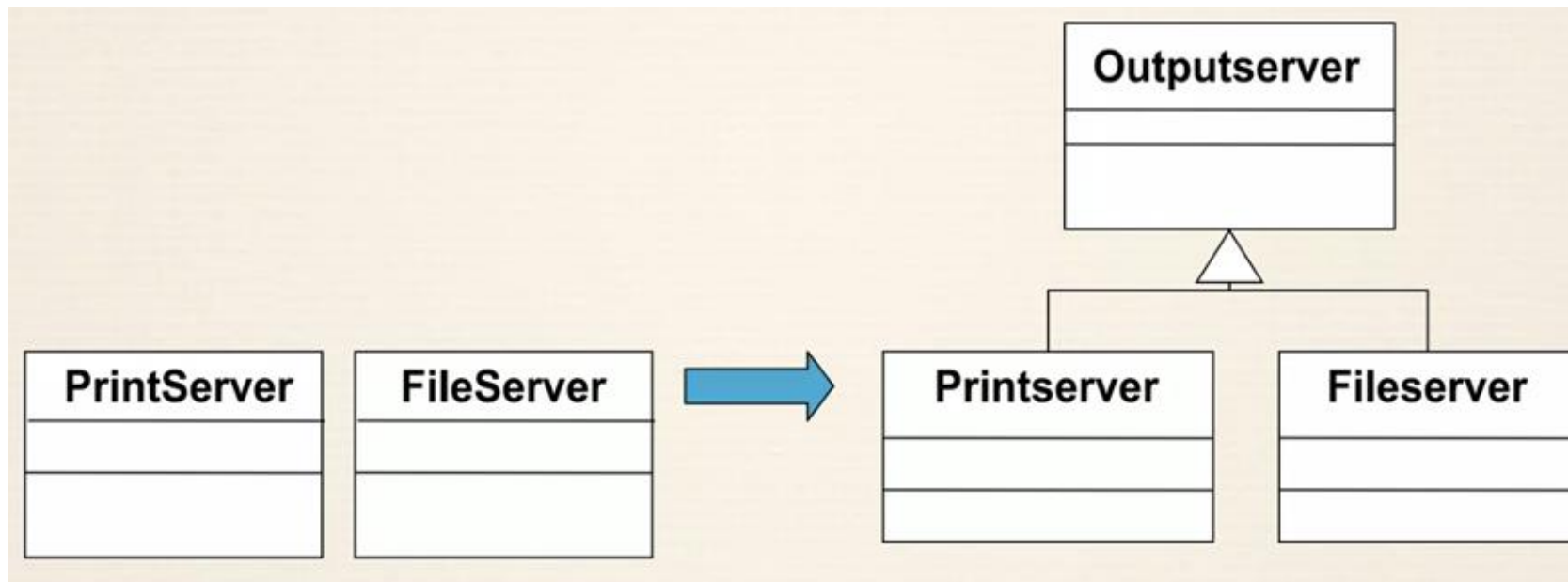


Extract superclass

- **Problem:** You have two classes with common fields and methods
- **Solution:** Create a shared superclass for them and move all the identical fields and methods to it



Extract superclass - Exercise



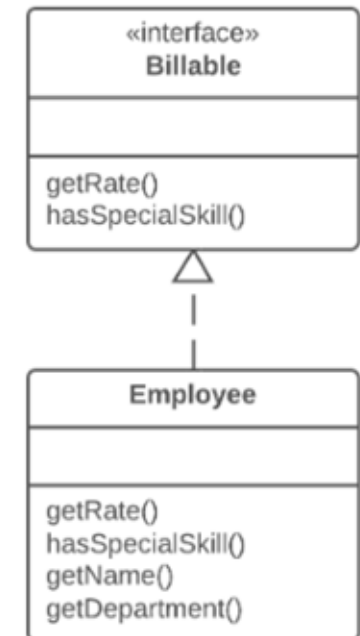
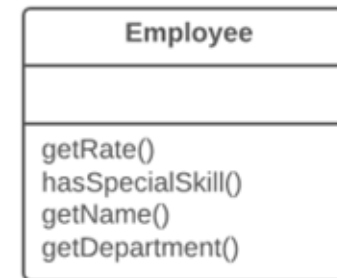
Extract interface

- **Problem**

- Multiple clients are using the same part of a class interface
- Another case: part of the interface in two classes is the same

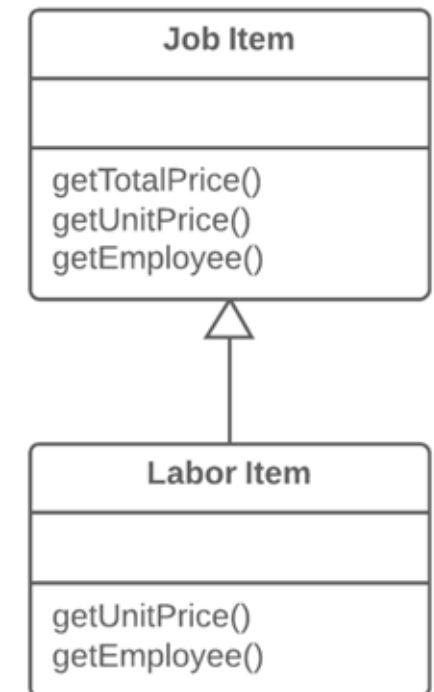
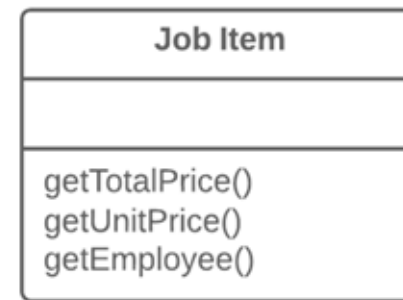
- **Solution**

- Move this identical portion to its own interface



Extract subclass

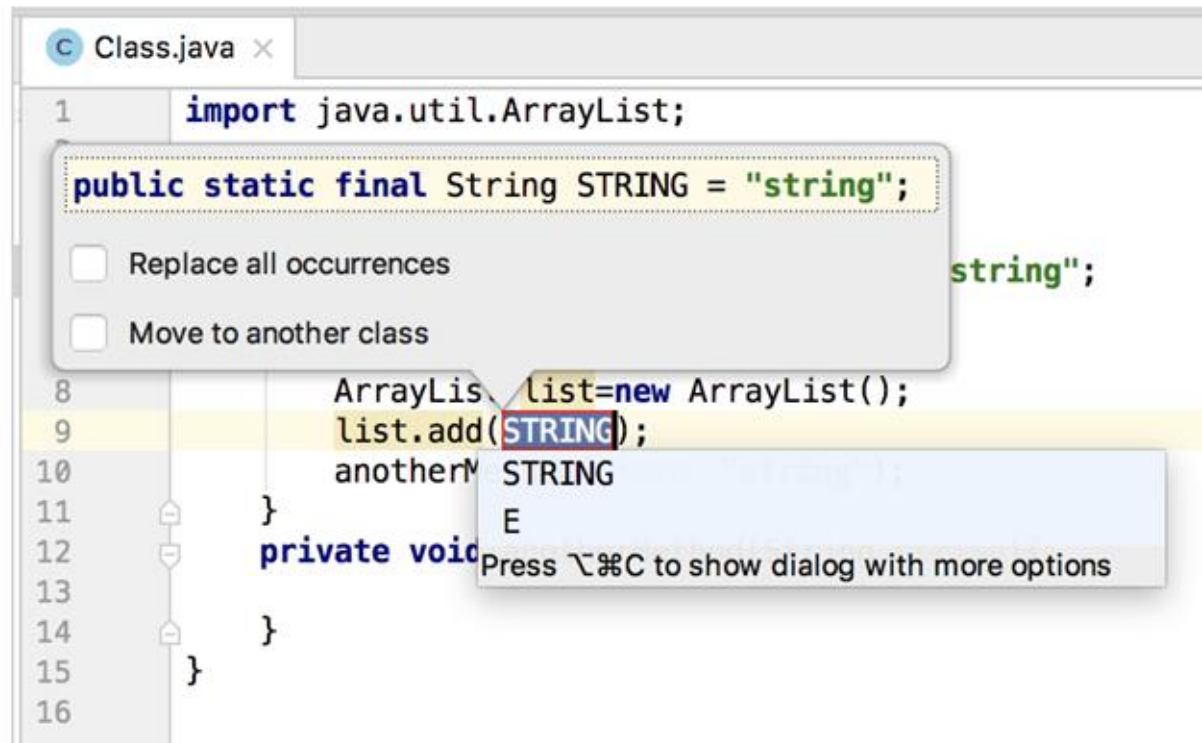
- **Problem:** A class has features that are used only in certain cases
- **Solution:** Create a subclass and use it in these cases



Tools

IntelliJ: <https://www.jetbrains.com/help/idea/refactoring-source-code.html>

Extract constant



```
public class Class {  
    public void method() {  
        ArrayList list = new ArrayList();  
        list.add("string");  
        anotherMethod("string");  
    }  
  
    private void anotherMethod(String string) {  
    }  
}
```

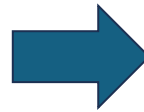


```
public class Class {  
    private static final String STRING = "string";  
  
    public void method() {  
        ArrayList list = new ArrayList();  
        list.add(STRING);  
        anotherMethod(STRING);  
    }  
  
    private void anotherMethod(String string) {  
    }  
}
```

<https://www.jetbrains.com/help/idea/extract-constant.html>

Extract field

```
public class Class {  
    AnotherClass anotherClass;  
    public void method() {  
        int a = 1;  
  
        int b = a + anotherClass.intValue();  
        int c = b + anotherClass.intValue();  
    }  
}
```



```
public class Class {  
    public AnotherClass anotherClass;  
    private int number;  
  
    public void method() {  
        int a = 1;  
        number = anotherClass.intValue();  
        int b = a + number;  
        int c = b + number;  
    }  
}
```

References

1. “Refactoring: Improving the Design of Existing Code” (2nd edition), Martin Fowler, 2018
2. “Clean Code: A Handbook of Agile Software Craftsmanship”, [Robert C. Martin](#), 2008
3. “Code Complete” (2nd Edition), [Steve McConnell](#), 2004
4. “Industry experiences with large-scale refactoring”, James Ivers et al., 2022 (<https://doi.org/10.1145/3540250.3558954>)
5. <https://refactoring.guru/refactoring>
6. <https://www.industriallogic.com/xp/refactoring/index.html>



Any Questions